The beauty of kNN

Readings for today

- Chapter 2: Statistical learning. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). An introduction to statistical learning: with applications in R (Vol. 6). New York: Springer.

- Chapter 4: Classification. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). An introduction to statistical learning: with applications in R (Vol. 6). New York: Springer.

Topics

1. kNN classification

2. kNN regression

kNN Classification

Fundamental regression classification problem

$$\begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} = f\left(\begin{pmatrix} x_{1,1} & \cdots & x_{1,p} \\ \vdots & & \vdots \\ x_{n,1} & \cdots & x_{n,p} \end{pmatrix}\right)$$

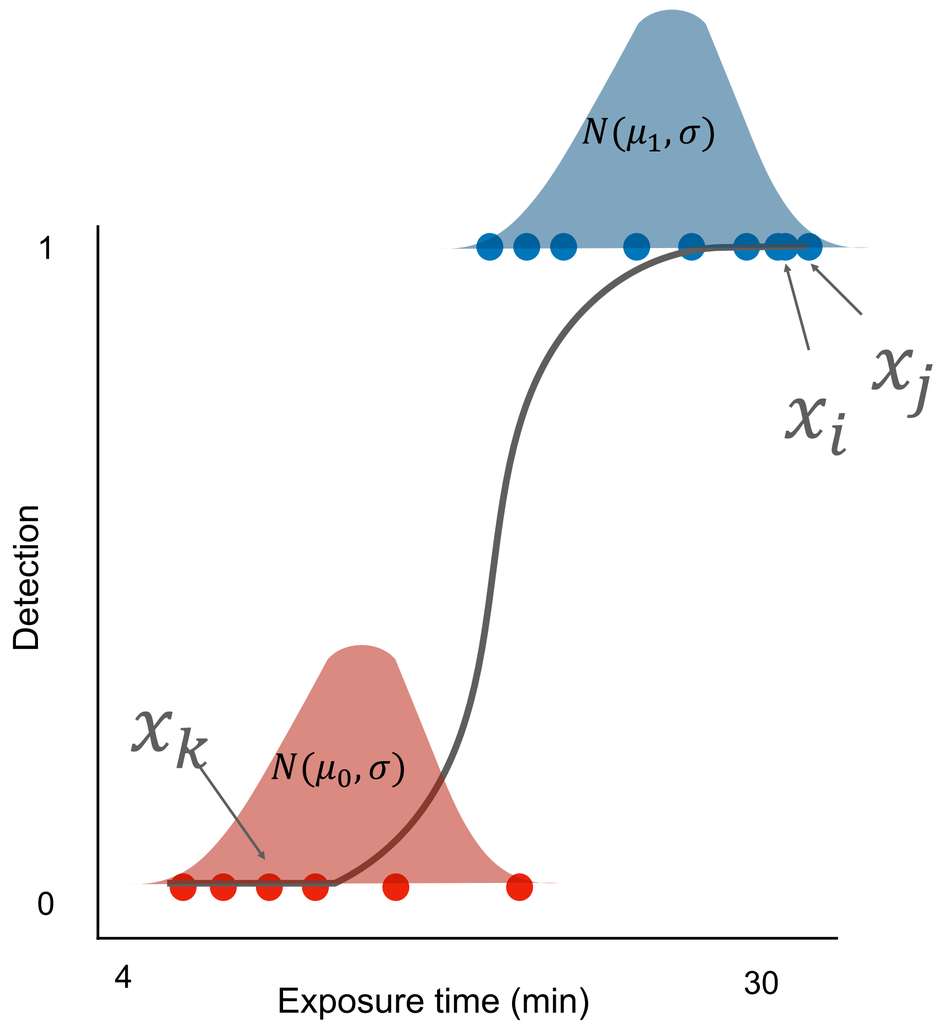
$$Y: \begin{cases} 1, & \text{if A} \\ 0, & \text{otherwise} \end{cases} \longrightarrow \text{categorical}$$

$$X \sim P(X \mid \beta) \longrightarrow \text{continuous}$$

$$P(Y = k \mid X = x_i) \leftarrow \text{Goal}$$

(James et al. 2013)

Nearest neighbors



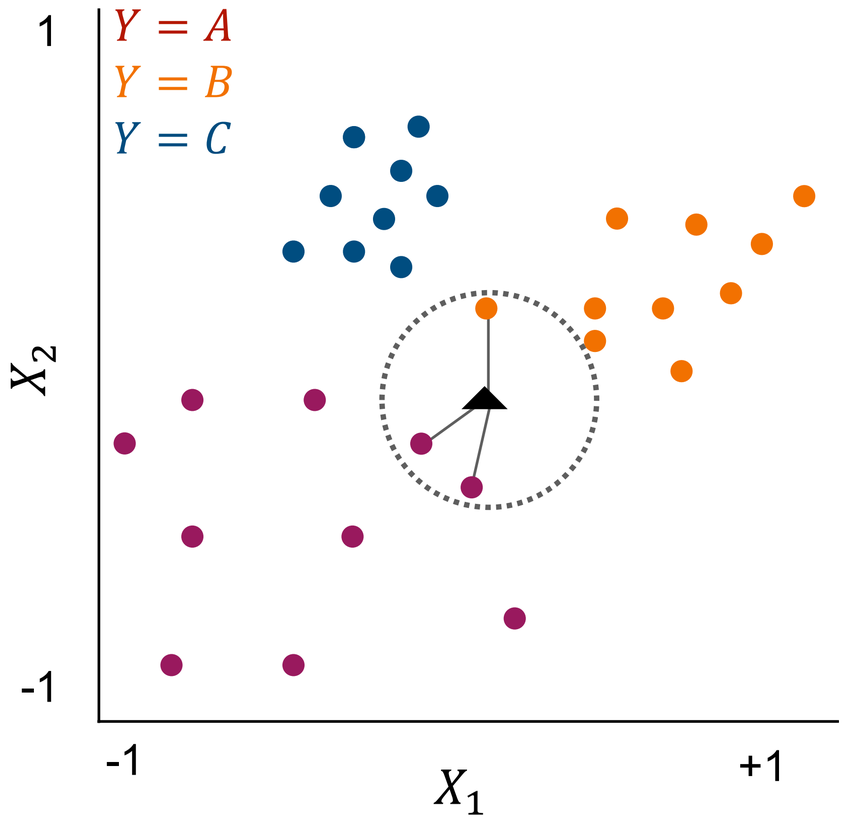
kNN on the other hand…

Observations in X that are closer together are likely to belong to the same group. No other assumptions required (i.e., non-parametric)

$$\downarrow distance = \uparrow likelihood$$

(James et al. 2013)

kNN classification



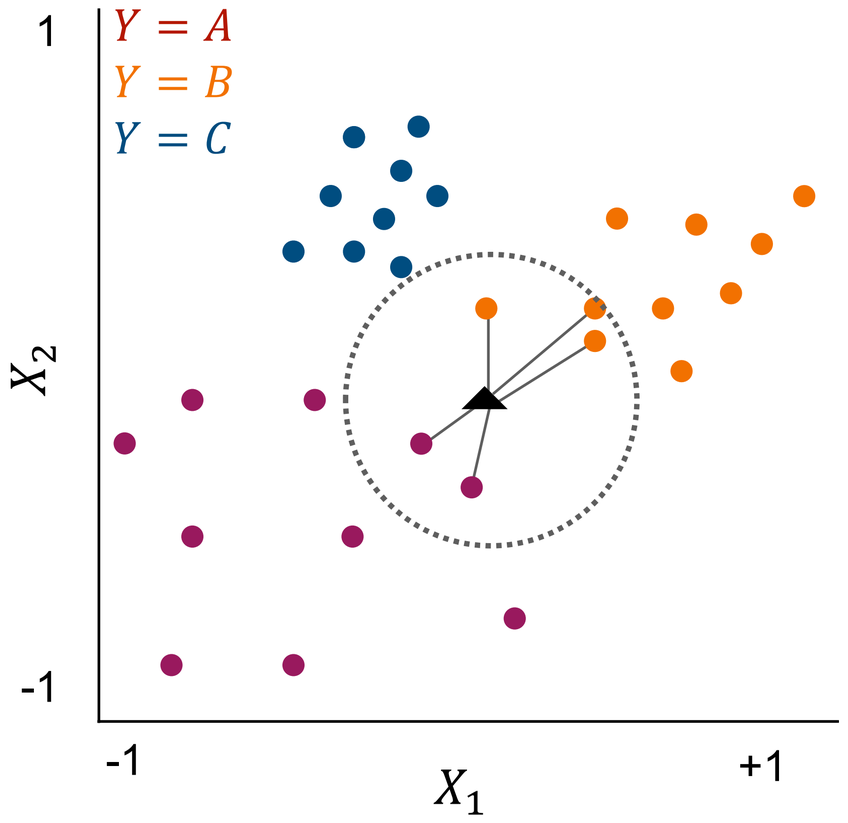
Euclidean distance: $d_{i,j} = \sqrt{\sum_{j=1}^{p} (x_{i,p} - x_{j,p})^2}$

k=3: (● ● ●) → A

Decision: Categorize by popular vote.

(James et al. 2013)

kNN classification



Euclidean distance: $d_{i,j} = \sqrt{\sum_{j=1}^{p} (x_{i,p} - x_{j,p})^2}$

k=3: (● ● ●) → A

k=5: (● ● ● ● ●) → B

Decision: Categorize by popular vote.

(James et al. 2013)

kNN classification algorithm

Step 1: Choose k.

Step 2: For every target observation x_i calculate all $d_{i,j}$.

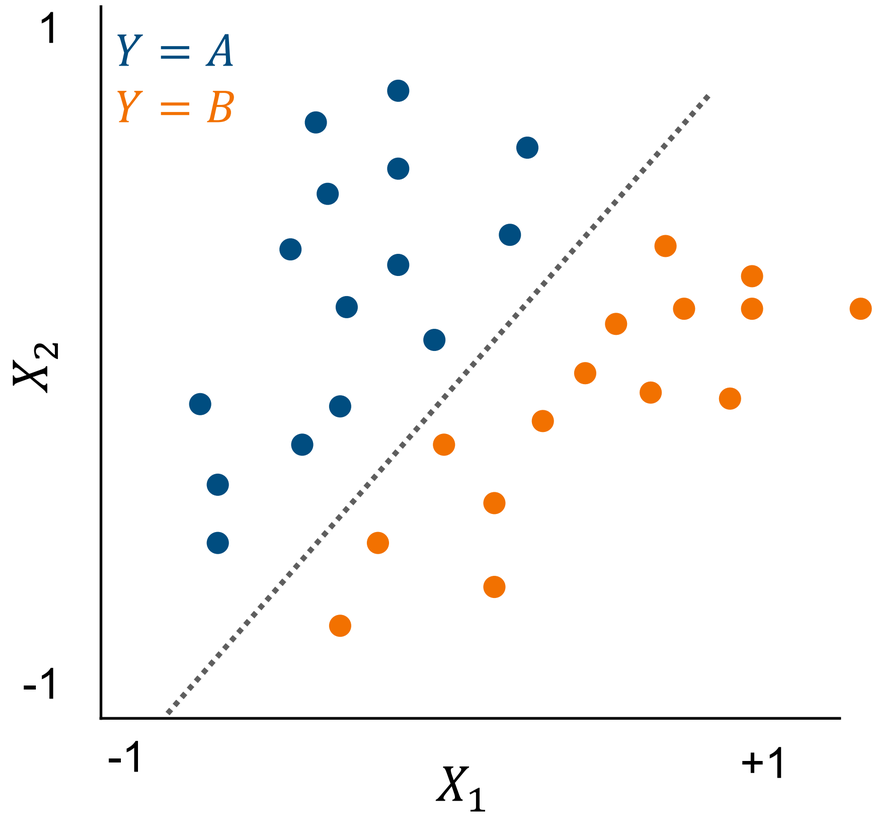
Step 3: Sort all $d_{i,j}$'s and select the k closest to x_i

Step 4: Categorize based on modal class in Step 3.

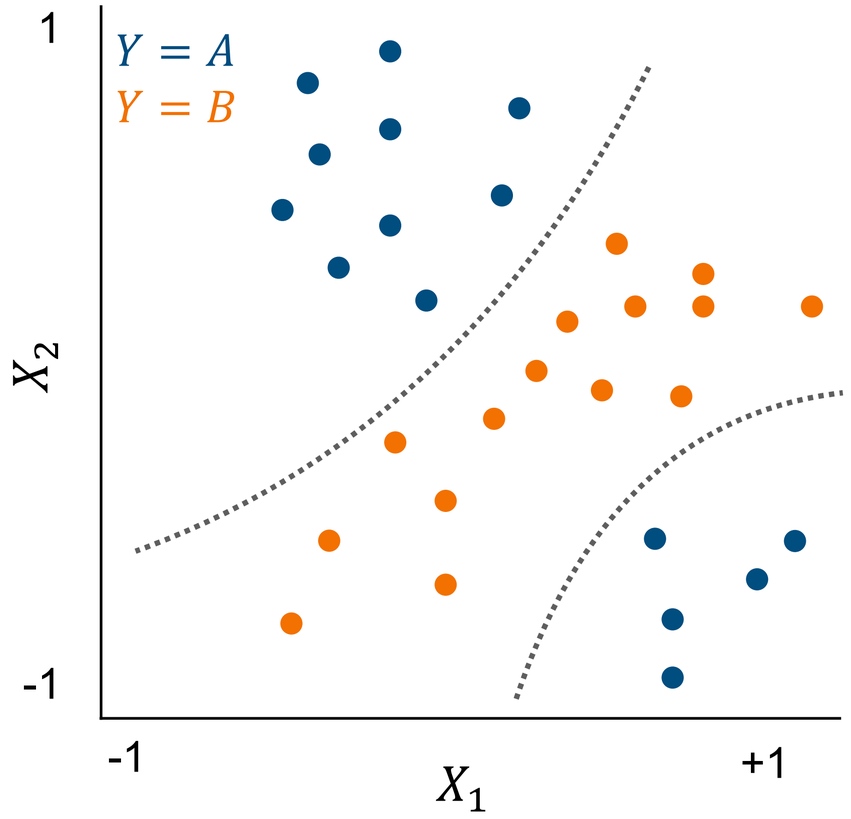
(James et al. 2013)

Decision boundaries

Logistic regression: linear boundary

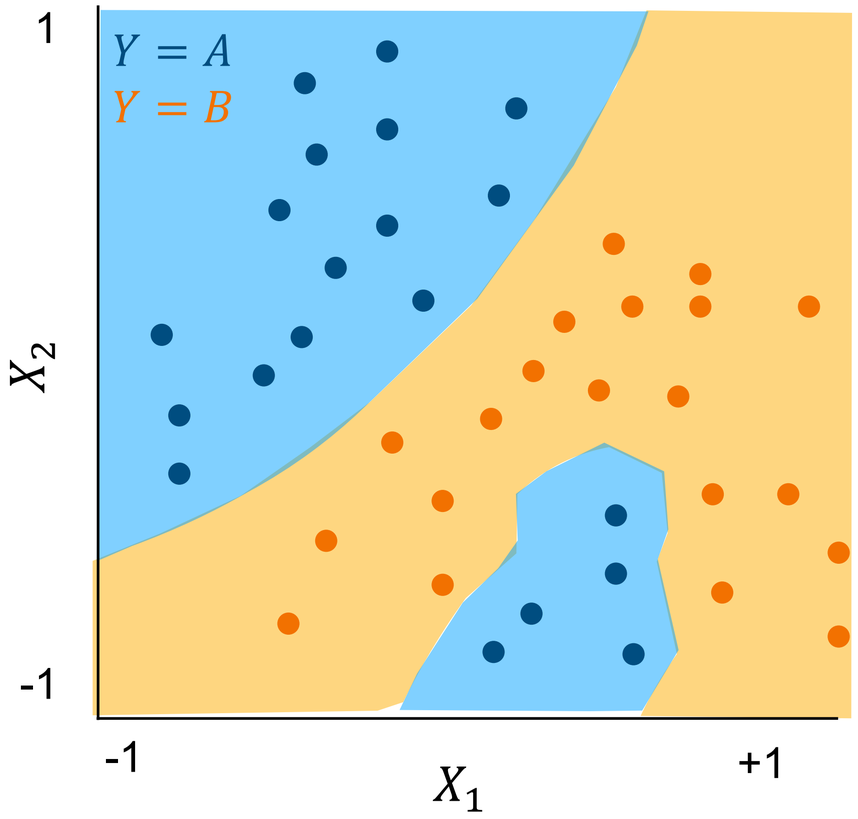


kNN: non-linear boundary



(James et al. 2013)

Defining territories via brute search



Territories

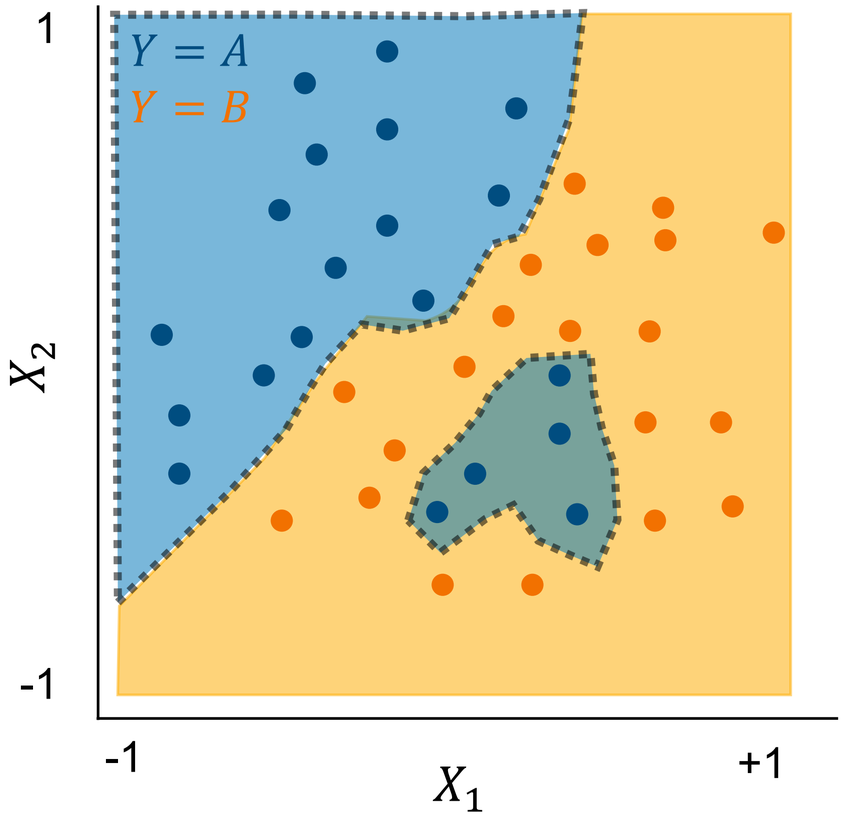
Iteratively search through all possible values of X (i.e. $[x_{min}, x_{max}]$) and use kNN to classify any possible state of X.

Decision Boundaries:

Positions in X where the vote is an exact tie.

(James et al. 2013)

Bias-variance tradeoff



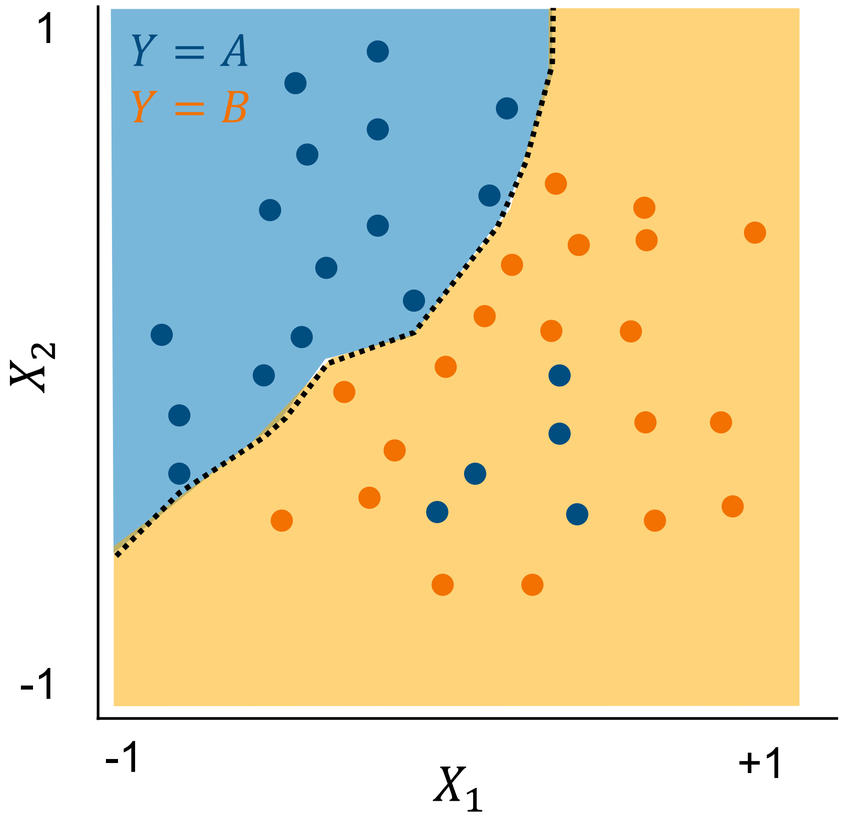
$$\uparrow k = \downarrow variance$$

k=1

- ↑ flexibility
- “islands” of group clusters

(James et al. 2013)

Bias-variance tradeoff



$$\uparrow k = \downarrow variance$$

k=1

- ↑ flexibility
- “islands” of group clusters

k=25

- clear segmentation
- higher error rate in training data

(James et al. 2013)

Prediction with kNN classifier

Full dataset

$$\begin{pmatrix} y_1 \\ \cdot \\ y_m \\ y_{m+1} \\ \cdot \\ \cdot \\ \cdot \\ y_n \end{pmatrix} = f\left(\begin{pmatrix} x_{1,1} & \cdots & x_{1,p} \\ \cdot & & \cdot \\ x_{m,1} & \cdots & x_{m,p} \\ x_{m+1,1} & \cdots & x_{m+1,p} \\ \cdot & & \cdot \\ \cdot & & \cdot \\ \cdot & & \cdot \\ x_{n,1} & \cdots & x_{n,p} \end{pmatrix} \right)$$

Test set

$$\begin{pmatrix} \hat{y}_1 \\ \cdot \\ \hat{y}_m \end{pmatrix} = \hat{f}_{train}\left(\begin{pmatrix} x_{1,1} & \cdots & x_{1,p} \\ \cdot & & \cdot \\ x_{m,1} & \cdots & x_{m,p} \end{pmatrix} \right)$$

Training set

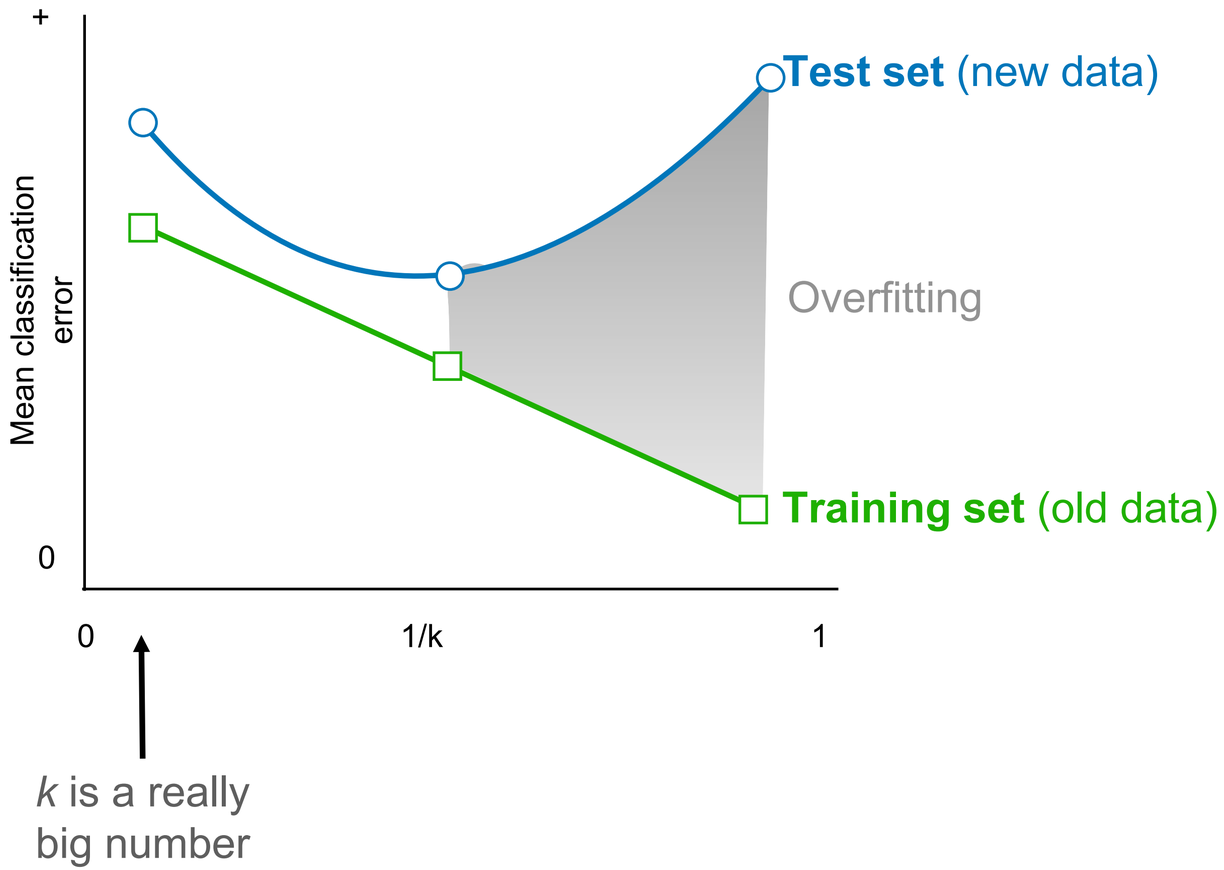
$$\begin{pmatrix} \hat{y}_{m+1} \\ \cdot \\ \cdot \\ \cdot \\ \hat{y}_n \end{pmatrix} = \hat{f}_{train}\left(\begin{pmatrix} x_{m+1,1} & \cdots & x_{m+1,p} \\ \cdot & & \cdot \\ \cdot & & \cdot \\ \cdot & & \cdot \\ x_{n,1} & \cdots & x_{n,p} \end{pmatrix} \right)$$

Prediction: $\hat{y}_i^{test} = \hat{f}(X_i^{test}, [X^{train}, Y^{train}])$

$For\ each\ value\ of\ k$

(James et al. 2013)

Bias-variance tradeoff



$$\uparrow k = \downarrow variance$$

(James et al. 2013)

kNN Regression

Classification vs. regression with kNN

1. The classification problem:

$$\hat{y}_i = \hat{f}(x_i) = P(Y = k|X = x_i) = \frac{1}{m}\sum_{l \in m} I(y_l = k)$$

$$I(y_i = k) = \begin{cases} 1, & in\ k \\ 0, & otherwise \end{cases}$$

2. The regression problem:

$$\hat{y}_i = \hat{f}(x_i) = P(Y = k|X = x_i) = \frac{1}{m}\sum_{l \in m} y_l$$

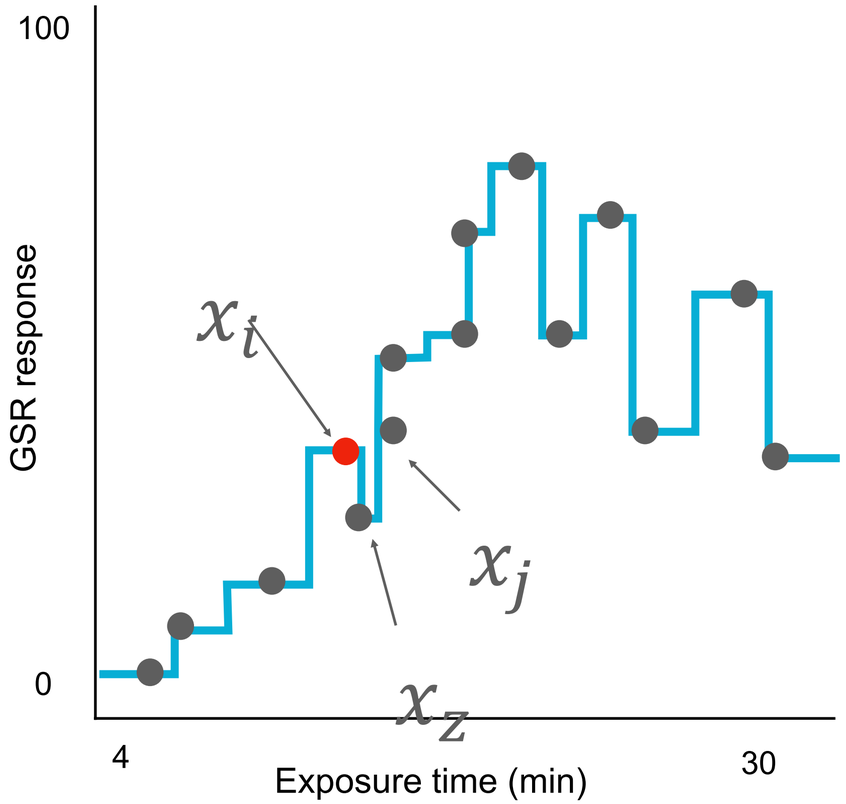
Regression with kNN is a classification problem where every value of y is its own unique category.

(James et al. 2013)

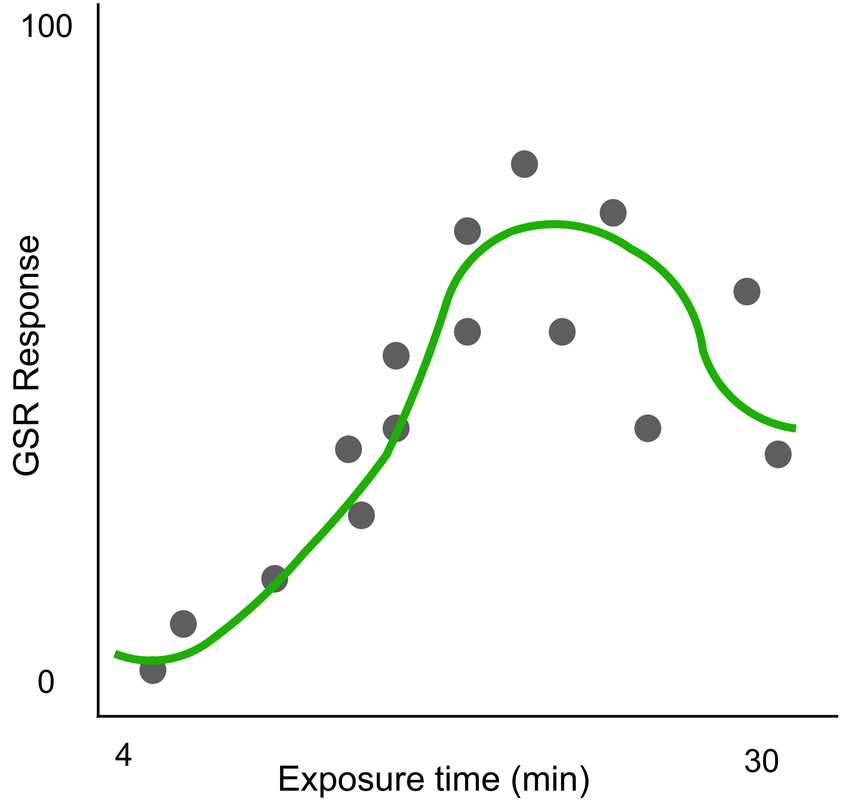
kNN regression

$\uparrow k = \downarrow variance$

k=1



k=25



$$When\ k > 1: \quad \hat{y}(x^*) = \frac{1}{k}\sum_{i=1}^{k} y(i)$$

(James et al. 2013)

Weighting kNN by distance

kNN classification

Each neighbor receives a **weight based on distance**, and the prediction is the **class with the largest total weight**.

$$\hat{y}(x^*) = \arg\max_c Score\ (c) \qquad Score(c) = \sum_{i \in c} w_i \qquad w_i = \frac{1}{d(x^*, x_{(i)})}$$

$distance\ ranges\ from\ [0, \infty)$

k=3 Neighbor	Distance	Class	Weight 1/d
1	0.1	A	10
2	0.9	B	1.11
3	1	B	1

kNN regression

The average is **weighted by distance**, giving closer points more influence.

$$\hat{y}(x^*) = \frac{\sum_{i=1}^{k} w_i y(i)}{\sum_{i=1}^{k} w_i} \qquad w_i = \frac{1}{d(x^*, x_{(i)})}$$

Penalize distal points even more

$$w_i = \frac{1}{d(x^*, x_{(i)})^2}$$

Limitations of kNN

Computationally expensive

Prediction requires:
- computing distance to every training observation
- sorting distances
- selecting the nearest neighbors (or mean of nearest neighbors)

And if you are defining territories as a heuristic then computation of learning boundaries increases exponentially for each p

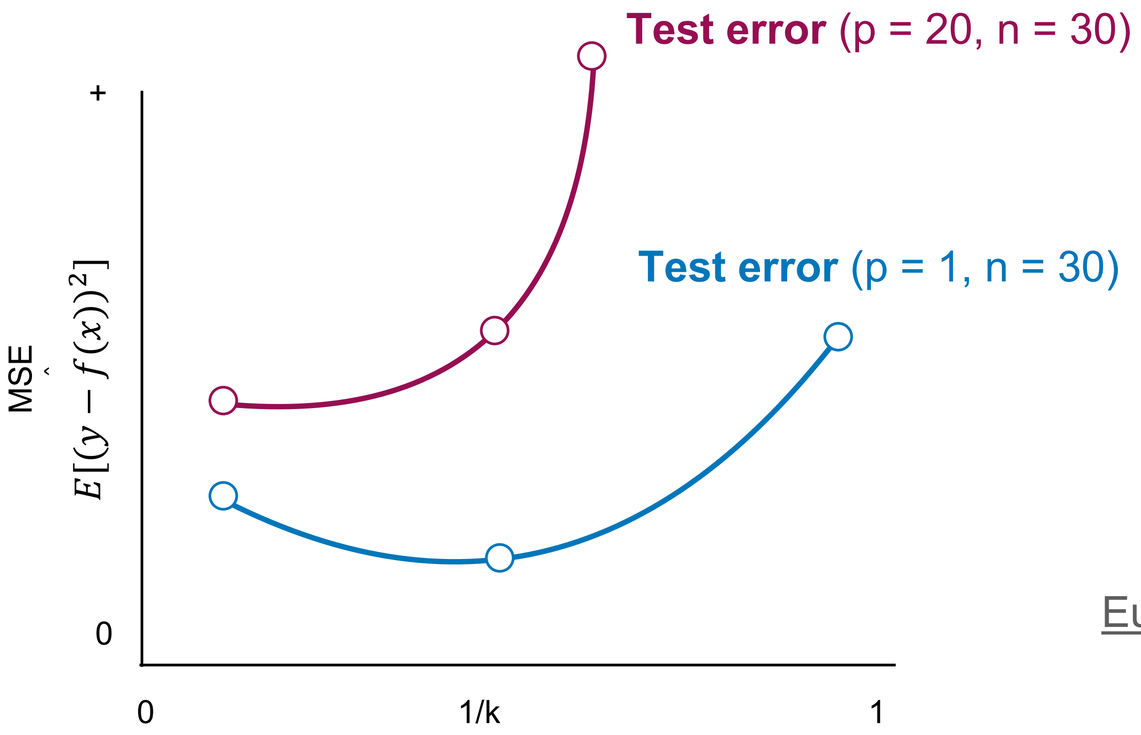
Limited inference

Great for prediction, but no easily interpretable parameters that communicate the relationship between X and Y.

Sensitive to dimensionality

There is a point where as p increases to a very large number kNN breaks down…

Curse of dimensionality



Problem:

As p increases, the distance become too sparse.

Euclidean distance

$$d_{i,j} = \sqrt{\sum_{j=1}^{p} (x_{i,p} - x_{j,p})^2}$$

Difference between farthest point and nearest point approaches 0

$$\frac{d_{max} - d_{min}}{d_{min}} \to 0$$

(James et al. 2013)

Avoiding the curse of dimensionality? RkCNN

Problem:

Low p : distances reflect structure
High p : all points $\approx$ equally far

Some subspaces will contain useful signals.

High-level Intuition

Full feature space: X: $\{x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8\}$

$\mathrm{Model}\ 1$: $\{x_1, x_3, x_5\}$ → compute kNN on Model 1

$\mathrm{Model}\ 2$: $\{x_2, x_4, x_7\}$ → compute kNN on Model 2

$\mathrm{Model}\ 3$: $\{x_1, x_6, x_8\}$ → compute kNN on Model 3

} Combine predictions into aggregate prediction of $\hat{y}$

(Lu et al. 2025)

RkCNN Algorithm

Step 1: Randomly sample features

- Randomly sample m features
- Repeat h times

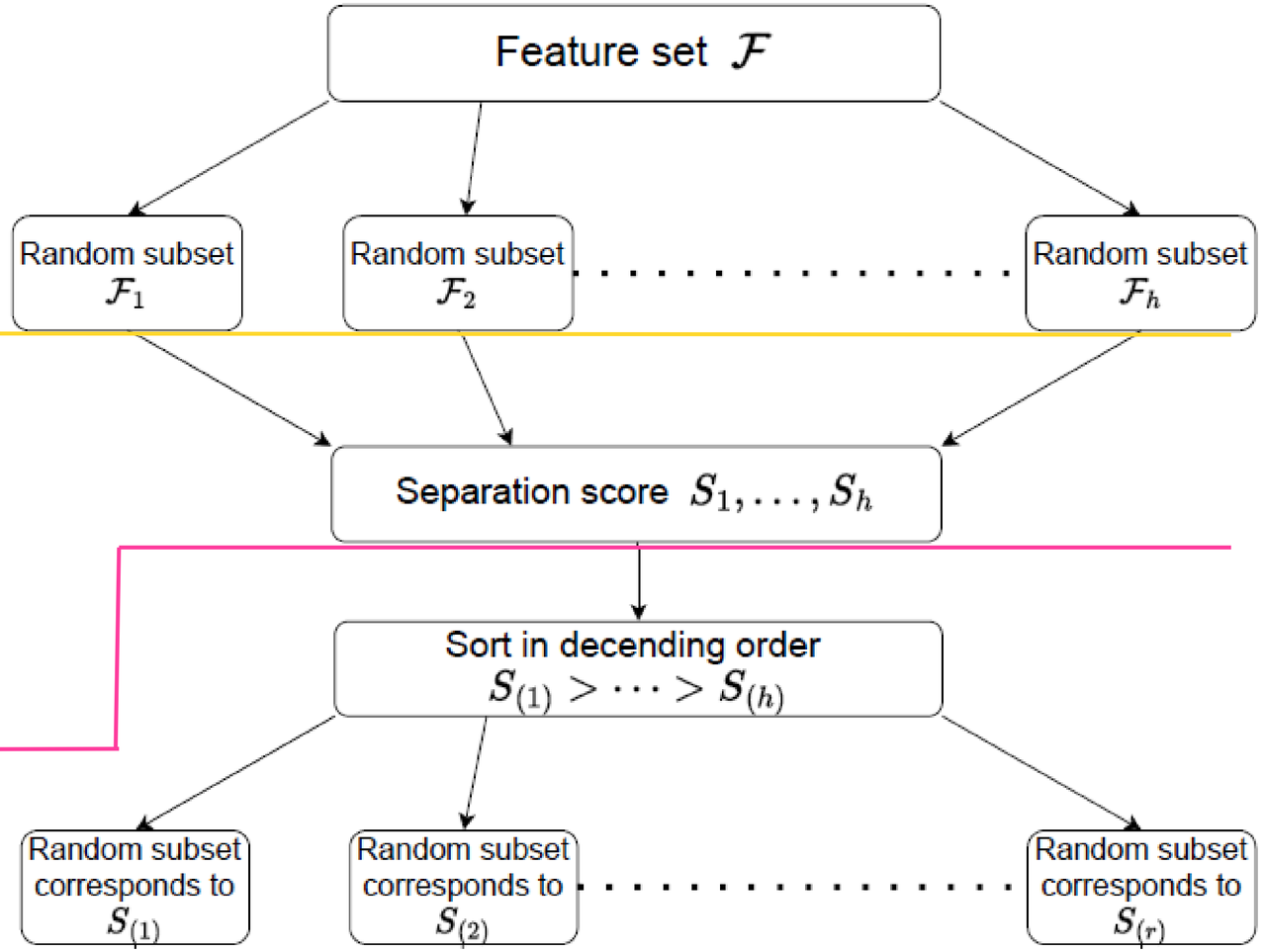
Step 2: Score how informative each subset is

$$S = \frac{between\ c\ variance}{within\ c\ variance}$$

High score → classes are separated in that feature subset
Low score → mostly noise

Step 3: Score how informative each subset is

Out of h subsets, keep top r with highest S score

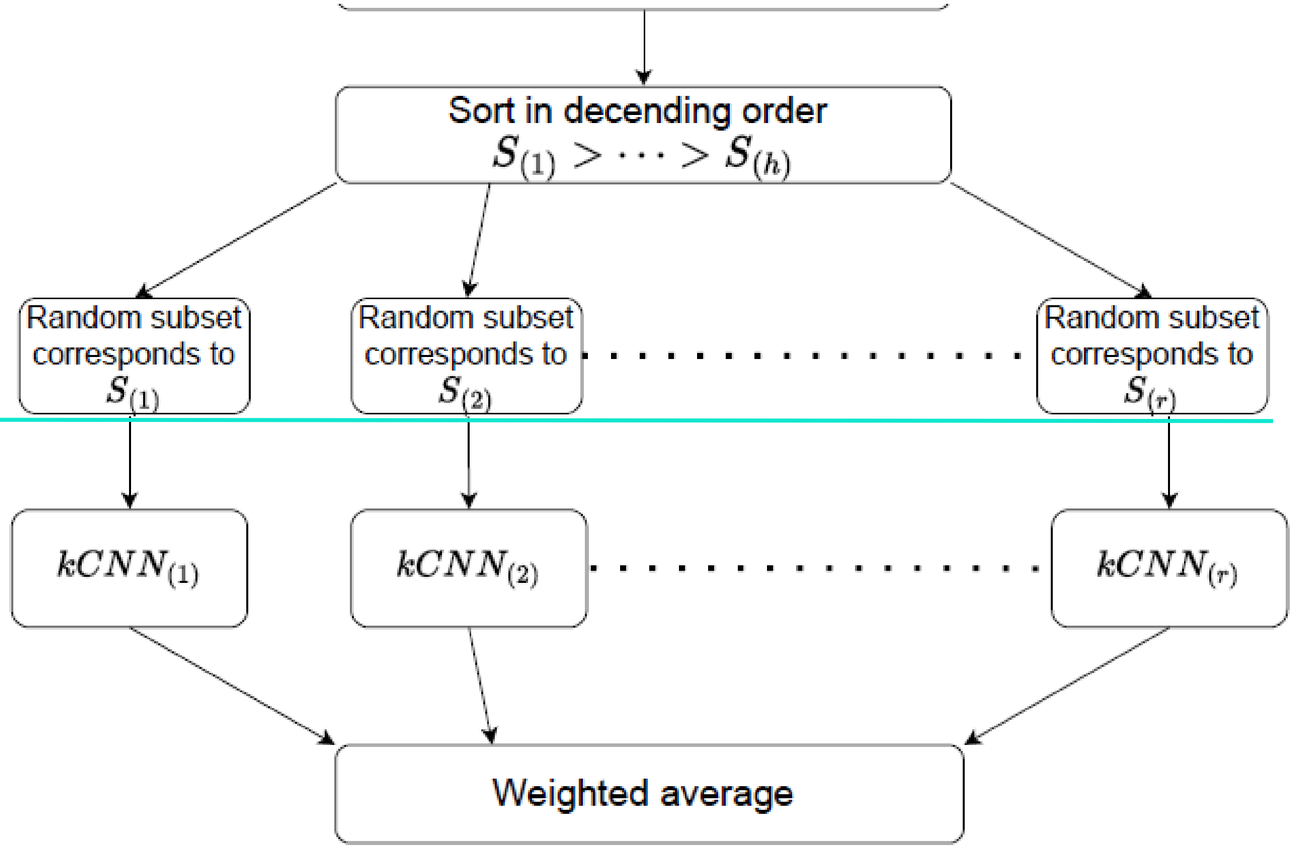


(Lu et al. 2025)

RkCNN Algorithm

Step 3: Score how informative each subset is

Out of h subsets, keep top r with highest S score



Step 4: Train kNN on each subset and aggregate

1. Train kNN for each r model and get the vote for class c from model j: $V_{j,c}(x)$

2. Weight each model vote by score from Step 2 & calculate prediction strength across classes:
   $prediction\ strength\ (c) = \sum_{j=1}^{r} w_j \cdot V_{j,c}(x) \qquad w_j = \frac{S_j}{\sum S}$

3. Use largest $prediction\ strength$ to make class prediction:
   $\hat{y} = argmax_c\ \ prediction\ strength(c)$

(Lu et al. 2025)

Take home message

- kNN offers a simple, non-parametric way to ask classification and regression questions in the prediction context.